CS 498: Machine Perception
HW4: Tiny NeRF for Novel View Synthesis

Fall 2026

Course weight	10%
Score	13 required points; +1 optional bonus
Work mode	Individual
Release	To be announced
Due	To be announced at 11:59 p.m. America/Chicago

1 What you will build

A photograph records one scene from one camera. In this homework, you will build a compact Neural Radiance Field (NeRF) that instead represents a 3D scene and synthesizes images from new camera poses. You will implement the complete path

camera pixels → world-space rays → sampled 3D points → neural color/density → rendered image.

You will then sample the learned density field and use a provided pipeline to extract, texture, and render a mesh. The assignment has six focused required code blocks. Camera-pose construction, network-layer construction, training, evaluation, marching cubes, mesh cleanup, texture baking, and mesh rendering are provided.

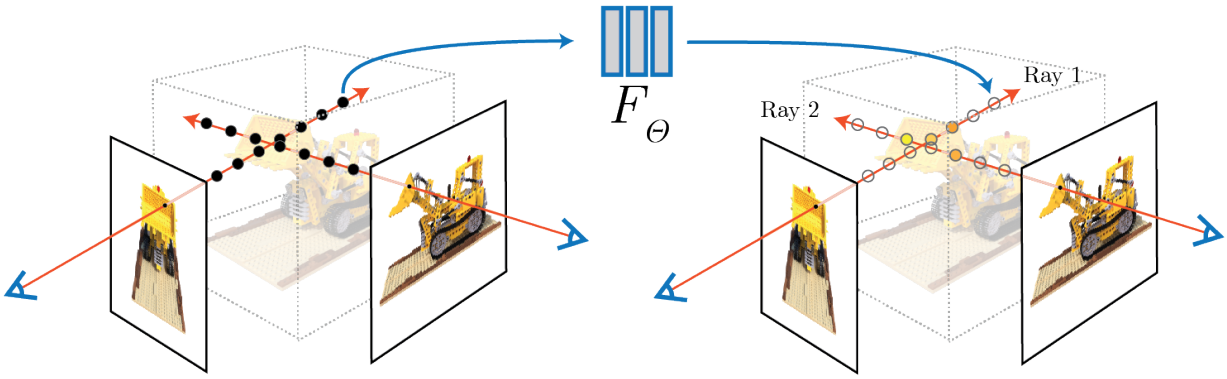


Figure 1: Conceptual NeRF pipeline: sample points on camera rays, evaluate a neural radiance field, and composite the samples into new views. Adapted from Mildenhall et al.

2 Run one file per task

From the distributed `hw4/` directory, create an environment, install dependencies, and download the public data:

```
conda create -n cs498-hw4 python=3.11
conda activate cs498-hw4
python -m pip install -r requirements.txt
python data/download_data.py
```

The downloader verifies the public archive and extracts 50 training, 5 validation, and 12 test images at 200×200 pixels under `data/nerf_synthetic/lego`. To recheck an existing installation, rerun the downloader with the `--verify-only` flag.

Complete the homework in order:

```
python task1_rays_encoding.py
python task2_nerf_rendering.py
python task3_train_mesh.py --mode check
python -m unittest discover -s tests
python task3_train_mesh.py --mode smoke
python task3_train_mesh.py --mode all
```

The untouched starter is expected to fail tests for unfinished blocks. The unit tests use small synthetic inputs and do not train a NeRF. The 100-step smoke run is for debugging only; final evidence must come from the 10,000-step run. On a recent desktop GPU, budget several minutes for the complete workflow; runtime varies by hardware.

Edit only code between `INSERT YOUR CODE BELOW` and `DO NOT MODIFY CODE OUTSIDE THIS BLOCK`.

File	Graded edits	Points
`task1_rays_encoding.py`	`get_rays`, `positional_encoding`	2.0
`task2_nerf_rendering.py`	`TinyNeRF.forward`, `sample_along_rays`, `raw_to_outputs`	4.5
`task3_train_mesh.py`	`build_density_grid`; provided training/evaluation pipeline	5.0
`report.pdf`	Methods, results, limitations, reproducibility	1.5

3 Representation contract

We use the NeRF/OpenGL camera convention: camera x points right, camera y points up, and the camera looks along negative z. Image rows increase downward, which explains the negative image-row term below. Every supplied pose is a camera-to-world transform. Ray directions remain unnormalized until volume rendering so that ray-parameter intervals can be converted to world-space distances.

Quantity	Shape in the full pipeline	Meaning
Ray origins/directions	`[H,W,3]`	world-space ray geometry
Encoded points	`[...,51]`	raw xyz plus 8 frequency bands
Encoded directions	`[...,27]`	raw direction plus 4 bands
Depths / sample points	`[N,64]` / `[N,64,3]`	ordered samples on each ray
Raw network output	`[N,64,4]`	RGB logits followed by density
Weights	`[N,64]`	per-sample volume-rendering contribution
RGB / depth / opacity	`[N,3]` / `[N]` / `[N]`	rendered ray outputs

4 Task 1: Rays and positional encoding [2.0 points]

1.1 Camera rays [1.0 point]

Implement `get_rays(height,width,focal,c2w)`. For pixel column i, row j, and focal length f in pixels, form

$$\mathbf{d}^c_{j,i} = [(i - W/2)/f,\ -(j - H/2)/f,\ -1]^\mathsf{T}.$$

For camera-to-world transform

$$\mathbf{T}_{c2w} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}, \qquad \mathbf{o}_{j,i} = \mathbf{t}, \qquad \mathbf{d}_{j,i} = \mathbf{R}\mathbf{d}^c_{j,i}.$$

Return origins and directions with shape `[H,W,3]`. Preserve the input floating dtype and device. Do not normalize the directions.

1.2 Positional encoding [1.0 point]

Implement `positional_encoding`. For $\mathbf{v} \in \mathbb{R}^D$ and L log-spaced frequency bands,

$$\gamma_L(\mathbf{v}) = \left[\mathbf{v}, \sin(2^0\mathbf{v}), \cos(2^0\mathbf{v}), \ldots, \sin(2^{L-1}\mathbf{v}), \cos(2^{L-1}\mathbf{v})\right].$$

The operations are componentwise. Preserve leading dimensions and concatenate only on the last axis. With raw input included, the encoded width is $D(1+2L)$. Support $L = 0$ and the supplied `include_input` and `log_sampling` options.

Run `pythontask1_rays_encoding.py`. It writes a three-panel diagnostic to `outputs/task1/task 1_visual_checkpoint.png`. The first two panels show ray-direction colors for two camera poses. Each should vary smoothly across the image; changing the pose should rotate the pattern without creating seams, flips, or isolated pixels. The final panel plots positional-encoding channels over a one-dimensional input sweep. Low-frequency channels should vary slowly, while successive higher-frequency sine/cosine pairs oscillate more rapidly.

5 Task 2: Tiny NeRF and volume rendering [4.5 points]

2.1 Network forward pass [2.0 points]

The supplied constructor creates six point layers of width 128. Implement only `TinyNeRF.forward`. Apply ReLU after each point layer and concatenate the original 51-D position encoding before point layer 3. Predict density from the final point feature. For color, transform the same feature, concatenate the 27-D encoded view direction, apply the supplied view layer and ReLU, and then the RGB head. Return raw values in RGB–density order:

$$f_\theta(\gamma_x, \gamma_d) = [c^r_{raw}, c^g_{raw}, c^b_{raw}, \sigma_{raw}].$$

Do not apply sigmoid or density ReLU inside the network; the renderer owns those activations.

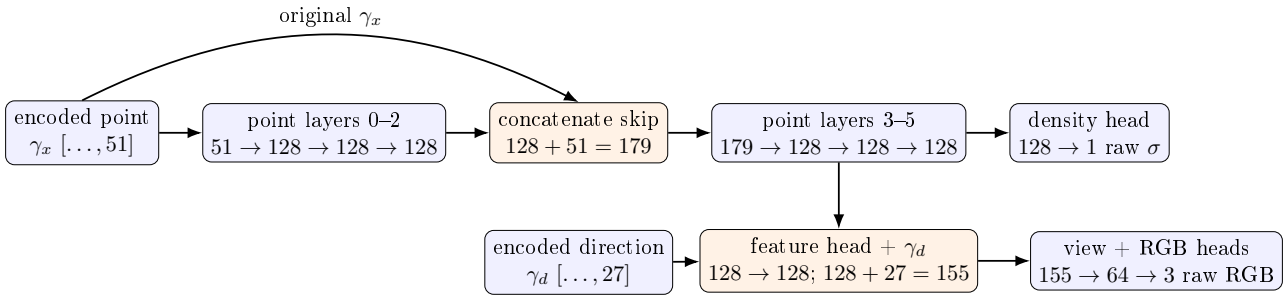


Figure 2: Required TinyNeRF forward architecture. The original position encoding enters again at the layer-3 skip. Density is predicted before view conditioning; only RGB depends on the encoded viewing direction. The returned tensor concatenates raw RGB with raw density to obtain [..., 4].

Stage	Input width	Output width
Point MLP layers 0–2	51 then 128	128
Skip and point layer 3	128 + 51 = 179	128
Point MLP layers 4–5	128	128
Density head	128	1 raw density
Feature plus direction	128 + 27 = 155	64
RGB head	64	3 raw colors

2.2 Stratified ray sampling [1.0 point]

Implement `sample_along_rays`. For S samples,

$$\bar{z}_s = z_{near} + \frac{s}{S-1}(z_{far} - z_{near}), \qquad s = 0, \dots, S-1.$$

When perturbation is enabled, construct strata from adjacent midpoints and sample independently inside each stratum. Return depths `[N,S]` and points $\mathbf{x}_{n,s} = \mathbf{o}_n + z_{n,s}\mathbf{d}_n$ with shape `[N,S,3]`. Keep all operations on the ray dtype and device.

For adjacent base depths, first form

$$m_s = \tfrac{1}{2}(\bar{z}_s + \bar{z}_{s+1}).$$

The first stratum starts at z_{near}, the last ends at z_{far}, and every interior boundary is one midpoint m_s. If ℓ_s and u_s are the lower and upper boundaries of stratum s, draw $z_{n,s} = \ell_s + \epsilon_{n,s}(u_s - \ell_s)$ with independent $\epsilon_{n,s} \sim \mathcal{U}[0,1)$. The nonoverlapping strata already keep the sampled depths ordered; do not sort them afterward.

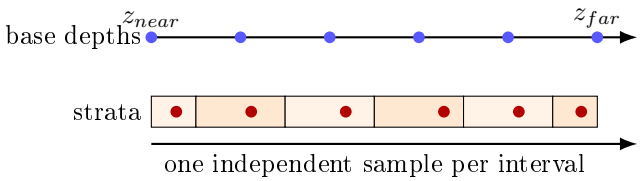


Figure 3: Deterministic samples define midpoint-bounded strata. Perturbation moves each sample only inside its own interval, independently for every ray.

Implementation recipe.

1. Create one `torch.linspace` of shape `[S]` on the ray device and dtype, then broadcast it to `[N,S]`.
2. If perturbing, build lower and upper arrays from the adjacent midpoints and draw a `[N,S]` random tensor.
3. Form points with two singleton dimensions: `rays_o[:,None,:]+rays_d[:,None,:]*z_vals[...,None]`.
4. Check that both outputs stay on the input device/dtype and that every perturbed value remains within $[z_{near}, z_{far}]$.

2.3 Alpha compositing [1.5 points]

Implement `raw_to_outputs`. Activate color and density as $\mathbf{c} = \mathrm{sigmoid}(\mathbf{c}_{raw})$ and $\sigma = \mathrm{ReLU}(\sigma_{raw})$. For $\delta_{n,s} = \Delta z_{n,s}\|\mathbf{d}_n\|_2$, append a large final interval and compute

$$\alpha_{n,s} = 1 - e^{-\sigma_{n,s}\delta_{n,s}}, \qquad T_{n,s} = \prod_{j<s}(1 - \alpha_{n,j} + 10^{-10}), \qquad w_{n,s} = T_{n,s}\alpha_{n,s}.$$

The prefix product is exclusive: sample s does not attenuate itself. Return

$$\mathbf{C}_n = \sum_s w_{n,s}\mathbf{c}_{n,s}, \quad D_n = \sum_s w_{n,s}z_{n,s}, \quad A_n = \sum_s w_{n,s},$$

plus weights and activated sample colors. On a white background, add $(1 - A_n)\mathbf{1}$ to $\mathbf{C}_n$. Keep the computation vectorized.

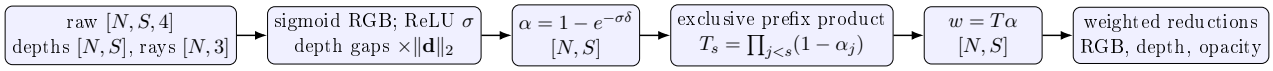


Figure 4: Front-to-back volume-rendering flow. Transmittance is exclusive: the opacity of sample s changes later samples, not its own incoming light.

Implementation recipe.

1. Split `raw[...,:3]` from `raw[...,3]` and apply the two activations before any reduction.
2. Subtract adjacent depths, append a 10^{10} final gap, and multiply every ray's gaps by its direction norm.
3. Compute α. For T, prepend a column of ones before `torch.cumprod`, then remove the extra final column. This shift creates the required exclusive product. Avoid a Python loop over samples; `torch.cumprod` can form the exclusive prefix product.
4. Form $w = T\alpha$ and reduce only across the sample dimension. Do not normalize weights to sum to one.
5. If the background is white, add the remaining transparency $1 - A_n$ to each RGB channel, not to depth or opacity.

Run `pythontask2_nerf_rendering.py`. It writes a four-panel diagnostic to `outputs/task2/task2_visual_checkpoint.png`. Deterministic sample depths should be identical across rays and increase evenly from near to far. Perturbed samples should vary across rays but remain ordered and inside their strata. In the compositing panels, the near-red example should concentrate its weight and rendered color near the early red sample; the far-blue example should do the same at the later blue sample; and the transparent example should have negligible weights and render the white background.

6 Task 3: Training, held-out views, and mesh [5.0 points]

3.1 Training and held-out reconstruction [3.0 points]

Run the API check and CPU smoke job before the full pipeline:

```
python task3_train_mesh.py --mode check
python task3_train_mesh.py --mode smoke
python task3_train_mesh.py --mode all
```

The full configuration is fixed:

Iterations	10000	Batch size	1024 rays
Point/direction bands	8 / 4	Samples per ray	64
Network	6 layers, width 128	Skip layer	3
Learning rate	5×10^{-4}	Validation/test views	5 / 12

The provided trainer minimizes RGB mean squared error and logs PSNR $= -10 \log_{10}(\text{MSE})$. It saves milestone previews at steps 2,500, 5,000, 7,500, and 10,000, evaluates all five validation views, and renders a 12-view held-out turntable. PSNR and SSIM are diagnostic higher-is-better metrics, not hard grading cutoffs.

Inspect your own outputs before submission. The training curve should improve overall after its initial rise, though it need not be monotonic. Final milestone images should be recognizable; validation comparisons should align in pose and coarse geometry; and adjacent turntable views should change smoothly. Blur and loss of fine detail are expected from the compact model.

3.2 Density grid and provided mesh pipeline [2.0 points]

Implement `build_density_grid`. For bounds $[b_{lo}, b_{hi}]$ and resolution R, define

$$q_a = b_{lo} + \frac{a}{R-1}(b_{hi} - b_{lo}), \qquad \mathbf{g}_{a,b,c} = [q_a, q_b, q_c]^{\mathsf{T}}.$$

Use `torch.meshgrid(...,indexing="ij")` so the grid axis order is x, y, z. Flatten the R^3 points, query them in chunks with one fixed normalized direction, apply ReLU to raw density channel 3, and reshape to `[R,R,R]`. Disable gradients and temporarily use evaluation mode, restoring the model's previous training state afterward.

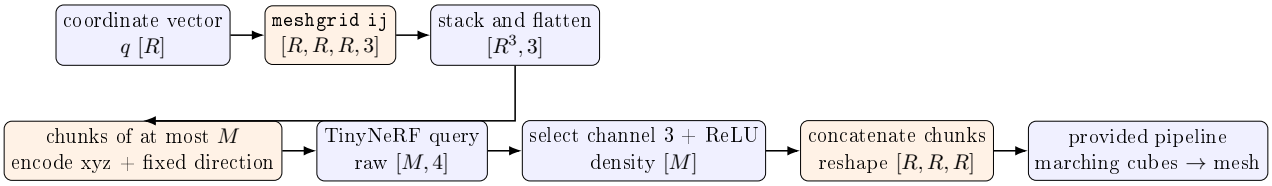


Figure 5: Density-grid data flow. Your function ends at the $[R, R, R]$ tensor; surface extraction, cleanup, texturing, export, and comparison are provided.

Implementation recipe.

1. Require $R \geq 2$ and a positive chunk size. Create coordinates on `device` using the model parameter dtype (or `float32` if the model has no parameters).
2. Stack the `ij`-indexed coordinate grids on the last axis and flatten to `[R**3,3]`; do not permute axes afterward.
3. Save `model.training`, switch to evaluation mode, and query under `torch.no_grad()`.
4. For each chunk, encode positions using `cfg["num_freqs_xyz"]`. Expand one fixed unit direction, such as $[0, 0, -1]$, and encode it using `cfg["num_freqs_dir"]`.
5. Collect `relu(raw[...,3])`, concatenate chunks in their original order, reshape to `[R,R,R]`, and restore the model's prior mode.

Typical mistakes are using `indexing="xy"`, mixing CPU and GPU tensors, querying all R^3 points at once, reading an RGB channel instead of channel 3, or forgetting to restore training mode.

The provided pipeline uses bounds $[-1.5, 1.5]^3$, resolution 96^3, density threshold 20, and a 256×256 texture atlas. It performs marching cubes, coordinate conversion, component cleanup, UV unwrapping, texture baking, OBJ/PLY export, and two matched-view comparisons. The mesh should preserve the main scene geometry and remain aligned with the two cameras. Holes or roughness around thin structures are acceptable; large unrelated floating shells or an unrecognizable object indicate a problem to fix before submission.

7 Optional bonus: NeRF-aware ball insertion [+1.0 point]

The analytic ray–sphere intersection is supplied. Implement only `composite_opaque_sphere`. For a sphere hit at $t_{b,n}$, keep samples strictly in front and compute

$$\mathbf{C}_{front,n} = \sum_{z_{n,s} < t_{b,n}} w_{n,s} \mathbf{c}_{n,s}, \qquad A_{front,n} = \sum_{z_{n,s} < t_{b,n}} w_{n,s}, \qquad T_{b,n} = \text{clamp}(1 - A_{front,n}, 0, 1).$$

For hit rays, return $\mathbf{C}_{insert,n} = \mathbf{C}_{front,n} + T_{b,n} \mathbf{c}_{ball,n}$; preserve the original NeRF RGB on misses. Return visibility $T_{b,n}$ on hits and zero on misses. Expected NeRF depth is not a valid z-buffer because it is a weighted average and does not represent partial foreground transmittance.

```
python bonus_ball_insertion.py --mode check
python -m unittest tests.test_bonus_ball_insertion
python bonus_ball_insertion.py --mode render \
  --checkpoint outputs/task3/lego_train/model.pt
```

The last command uses the locally generated model checkpoint. The checkpoint is an execution intermediate and is not part of the submission.

8 Report and submission [1.5 points]

Use `report_template.tex` as the report outline; a concise 4–6 page report is recommended. Include the Task 1 and Task 2 visual checkpoints; the training curve and final milestone previews; mean validation PSNR/SSIM and a held-out comparison; the turntable contact sheet; both mesh comparisons and mesh metrics; limitations; reproducibility commands/hardware; and the AI-use declaration. Include the optional insertion evidence only if attempted.

Submit one archive named `netid_hw4.zip` containing the following lightweight canonical files. Keep these paths unchanged.

- At the archive root: `task1_rays_encoding.py`, `task2_nerf_rendering.py`, `task3_train_mesh.py`, and `report.pdf`.
- Under `outputs/task1/`: `ray_directions.png` and `task1_visual_checkpoint.png`. Under `outputs/task2/`: `task2_visual_checkpoint.png`.
- Under `outputs/task3/lego_train/`: `train_metrics.csv`, `train_psnr_curve.png`, `val_010000_view_00.png`, `val_010000_view_01.png`, `validation_metrics.csv`, `turntable.mp4`, and `turntable_contact_sheet.png`.
- Under `outputs/task3/lego_train/validation/`: comparison images `view_00_comparison.png` through `view_04_comparison.png`.
- Under `outputs/task3/lego_train/mesh/`: `comparison.csv`, `view_00_comparison.png`, `view_01_comparison.png`, `lego_textured.obj`, `lego_textured.mtl`, `lego_texture.png`, and `lego_vertex_color_mesh.ply`.

If attempting the bonus, also submit `bonus_ball_insertion.py` and the ball MP4, contact sheet, and visibility image under `outputs/task3/lego_train/`. Do **not** submit model checkpoints (`*.pt`), smoke/debug outputs, data, environments, caches, or extra generated files. Training checkpoints may remain on your machine. Keep the listed visual and metric paths unchanged so the submission can be checked consistently.

9 Grading summary

Row	Full-credit evidence	Points
Rays and encoding	Smooth pose-dependent ray fields; correct multi-band encoding	2.0
Sampling and compositing	Ordered/bounded samples; correct opaque-sample behavior	2.5
Model and training	Correct model behavior; improving curve; recognizable milestones	2.0
Held-out reconstruction	Coherent comparisons, metrics, and turntable	3.0
Density and mesh	Recognizable geometry, aligned views, consistent mesh files/metrics	2.0
Report and reproducibility	Readable, complete, consistent with submitted outputs	1.5
Required total		13.0
Optional ball insertion	View-consistent foreground attenuation and visibility	+1.0
Maximum with bonus		14.0

10 Integrity and permitted tool use

This is an individual assignment. The required functions, analysis, figures, and report must be your own work. Generative-AI tools may not be used to write required code or report content unless the instructors explicitly announce a different policy for this offering. You may use course notes, cited public references, and ordinary debugging/documentation tools consistent with course policy. The instructors may ask you to explain any submitted implementation or result. Your code, report, numerical values, and generated outputs must agree.

Reference

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. ECCV, 2020.